Lively Interactive — Technical Operations

Unified Development Handbook: Code Isolation, SEO Strategy, Performance Targets, & Corrected Backend Security Controls
Version: 2.0 (Updated Baseline) | Date: October 2026

1. Code Isolation Framework (Prevention of Regression Bugs)

To ensure visual stability, a strict architectural firewall separates the immersive WebGL experience from user interface functionality.

- **Decoupled Background Execution:** The 3D Forest Path rendering engine must exist entirely within its own dedicated sandbox component or canvas context module. No application business logic, form handoffs, or navigation triggers may write directly to the active WebGL rendering pipeline loop.
- **UI Overlay Architecture:** All text content, brand messaging, user input elements, and navigation selectors must be written exclusively as native semantic HTML elements positioned above the canvas container using clean CSS layers (e.g., using absolute or fixed layout rules with distinct z-indexing).
- **State Protection Mandate:** Front-end developers executing copy updates or UI adjustments are strictly prohibited from editing scripts inside the WebGL initialization or loop files. Changes to textual content must never induce unexpected changes to 3D scenes, camera paths, or spatial structures.

2. Technical & Expansion SEO Architecture

To capture local enterprise clients in India and scale into global international markets, search engines must be capable of indexing the application cleanly.

- **Crawler Visibility:** Because search crawlers cannot parse text rendered inside WebGL meshes or graphic frames, all core copy, services overviews, and headers must live directly in the standard HTML DOM context.
- **Multi-Page Structure for Global Indexing:** Relying exclusively on single-page hash navigation prevents the site from ranking for specific keyword clusters. The core engine background must persist globally while standard front-end routing structures load dedicated content layers for targeted micro-pages: */services/whatsapp-automation* and */services/custom-software-development*.
- **Structured Metadata:** Ingress pipelines must inject comprehensive JSON-LD schemas representing Corporation data models, explicitly establishing local operational footprints alongside global target markets.

3. 60 FPS Performance Optimization Controls

Maximum smooth frame rate output must be protected across standard cellular network links.

- **Asset Optimization:** All 3D environment components, asset meshes, and scene models must run through Draco compression filters prior to deployment. Texture arrays must strictly employ lightweight compression

frameworks (.webp or Basis Universal asset styling).

- **Asynchronous Rendering Priorities:** Critical text blocks, styling structures, and layouts must compile and display immediately upon user landing. The initialization script of the background 3D canvas must run asynchronously to ensure the interface is active prior to heavy asset downloads finishing.

4. Hardened Production Security Controls (Corrected Architecture)

Integrating secure production pipelines is critical for international corporate evaluation. Below are the mandatory security rules, including a critical correction regarding data transport architectures.

CRITICAL SECURITY DIRECTIVE (CORRECTION OF COMMON ERROR):
CRITICAL ERROR CORRECTION: Never utilize public database keys or direct client-to-database connection configurations within the client-side front-end layer. Exposing internal direct database keys allows malicious actors to manipulate data fields via standard browser diagnostic tools. All transactional commands must route exclusively through an authenticated backend API proxy layer utilizing server-side environment variables (.env files).

4.1 App Secrets & Data Protection

- **API Credential Protection:** All high-level integration tokens (e.g., WhatsApp Enterprise keys, generative AI routing credentials) must be stripped from public code environments and hosted securely via isolated server-side environments.
- **Secure Persistence Protocols:** High-value client records captured via automated workflows must be encrypted at rest, and user account access credentials must be systematically mapped through industry-standard hashing protocols (such as bcrypt or Argon2).

4.2 Form Validation & Endpoint Hardening

- **Injection Interception:** All database manipulation strings must be parameterized. Front-end layers must execute rigid input type validation while server-side ingestion scripts filter out scripting vectors to block Cross-Site Scripting (XSS) and database injection hazards.
- **Row-Level Security (RLS):** Backend data integrations operating in tandem with front-end components must implement strict role boundaries and Row-Level Security parameters, rendering individual client records inaccessible to adjacent unauthenticated accounts.

4.3 Network Integrity & Lifecycle Management

- **Rate Limiting & Transport Encryption:** Inbound operational requests must be throttled by IP frequency to mitigate brute-force disruption vectors. Transport pipelines must explicitly mandate HTTPS configurations.
- **Dependency Vulnerability Auditing:** Automated auditing scripts must run continuous security checks during builds to catch security defects in third-party npm modules.
- **API Ingress Pruning:** Data routers must selectively map out database fields before transmission. Only fields explicitly required by the active component interface may be emitted to the client application layer.